Chapter II

Big Data Ecosystem & Technologies

Objectives

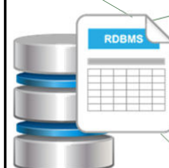
1. Understand the Big Data Ecosystem — Explain the evolution of Big Data technologies and why they are necessary.
2. Differentiate Between Batch and Stream Processing — Explain the difference between batch processing and stream processing

Section N° 1

Big Data With Traditional Data Storage

Recap of Classical RDBMS

Relational Database Management Systems (RDBMS) are software systems that store and manage structured data in tabular format.



Tables: Data is organized into rows and columns, with predefined structures.

Schema: RDBMS requires a well-defined schema that specifies table structures and relationships.

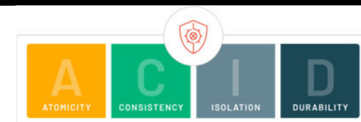
Atomicity: Transactions are all-or-nothing.
Consistency: Data remains in a valid state before and after transactions.
Isolation: Concurrent transactions do not interfere
Durability: Completed transactions are permanently stored.

Strengths
- Ideal for structured data with clear relationships.
- Ensures data reliability and integrity.
- Widely used in applications.

Why RDBMS Struggles with Big Data

Scalability Issues	- RDBMS typically relies on vertical scaling (adding more power to a single server) - Big data requires horizontal scaling (adding more servers)
Data Variety Limitations	RDBMS is optimized for structured data with predefined schemas. Cannot efficiently handle semi-structured or unstructured data.
Processing Velocity Challenges	RDBMS processes data in batches and struggles with high-velocity, real-time data streams (e.g., IoT or social media data).
Cost Inefficiency	Managing petabytes of data in an RDBMS is prohibitively expensive and operationally complex.

ACID Properties



Atomicity	Consistency	Isolation	Durability
Ensures that a transaction is treated as a single indivisible unit of work, meaning that either all its changes are committed or none of them are.	Guarantees that a database remains in a valid state before and after a transaction, enforcing integrity constraints and data integrity rules	Ensures that concurrent transactions do not interfere with each other, maintaining data integrity and preventing anomalies.	Guarantees that once a transaction is committed, its changes are permanent and will survive system failures.

+ Reliable and consistent data
+ Data integrity
+ Well-suited for complex transactions
+ Strong data protection

- Potential performance overhead
- Limited scalability

Big Data Challenges.

Too Big

Too Complex

Too Fast

Scalability	- Managing exponential growth of data volumes. - Ensuring seamless horizontal scaling
Infrastructure	- High costs big data infrastructure. - Ensuring high availability and fault tolerance
Privacy	- Ensuring compliance with regulations. - Protecting sensitive personal data
Quality	- Dealing with noisy, incomplete, or duplicate data. - Maintaining consistency across distributed datasets.

Distributed Data Storage As a Solution

Grace Brewster Hopper
1906-1992

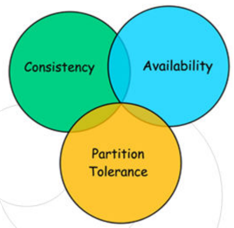
Computer scientist and Navy admiral. Pioneer of computer programming. First to devise the theory of machine-independent programming languages, and used it to develop COBOL.

"In pioneer days they used oxen for heavy pulling, and when one ox couldn't budge a log, they didn't try to grow a larger ox. We shouldn't be trying for bigger computers, but for more systems of computers"

CAP Theorem

Theorem - Eric Brewer in 2000

A distributed data system cannot simultaneously provide consistency, availability, and partition tolerance. It can only guarantee two of the three at any given time.



Consistency

Ensures that all nodes in a distributed system show the same view of the data at all times.

Availability

The system is consistently operational and can quickly handle read and write requests. It ensures users can access data and perform transactions even when some nodes are down

Partition Tolerance

Refers to the system's ability to function despite network partitions or disruptions in communication between different parts of the system.

BASE Properties

B A S E

Basically Available

It is the database's concurrent accessibility by users at all times. One user doesn't need to wait for others to finish the transaction before updating the record.

Soft state

Refers to the notion that data can have transient or temporary states that may change over time, even without external triggers or inputs.

Eventually consistent

Means the record will achieve consistency when all the concurrent updates have been completed. At this point, applications querying the record will see the same value.

ACID vs BASE

	ACID	BASE
Scale	Scales vertically.	Scales horizontally.
Flexibility	Less flexible. Blocks specific records from other applications when processing.	More flexible. Allows multiple applications to update the same record simultaneously.
Performance	Performance decreases when processing large volumes of data.	Capable of handling large, unstructured data with high throughput.
Synchronization	Yes. Adds delay when synchronizing.	No synchronization at the database level.

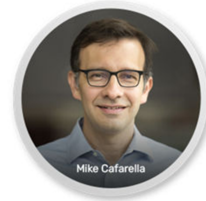
Section N° 2

Big Data Ecosystem

Big data Ecosystem refer to the **software** specifically designed to **analyze**, **process**, and **extract** information from **complex data sets**.

Apache Hadoop
2005

An Open Source framework that allows distributed processing of large data-sets across the cluster of commodity hardware



It was originally developed to support distribution for the Nutch search engine project. Doug, who was working at Yahoo! at the time and is now Chief Architect of Cloudera, named the project after his son's toy elephant. Cutting's son was 2 years old at the time and just beginning to talk. He called his beloved stuffed yellow elephant "Hadoop".

An **Open Source** framework that allows **distributed** processing of large data-sets across the **cluster** of **commodity hardware**.

A framework written in Java
Inspired by Google's Map-Reduce programming model as well as its file system (GFS)

1. Open Source
   - Source code is freely available
   - It may be redistributed and modified
2. Distributed Processing
   - Data is processed distributedly and independently on multiple nodes / servers
3. Cluster
   - Multiple machines connected together
   - Nodes are connected via LAN
4. Commodity Hardware
   - Economic / affordable machines
   - Typically low performance hardware

- 2002: Nutch Project was started by Doug Cutting and Mike Cafarella
- 2003: Google released white paper on GFS
- 2004: Google released white paper on Map Reduce
- Mid 2004: Nutch implemented NDFS and Map Reduce
- 2006: Development of Hadoop started as a sub-project of Nutch
- 2007: Yahoo started using Hadoop on 1000 node cluster
- 2008: Hadoop became top-level project at Apache
- Mid 2008: Hadoop defeated Supercomputers
- 2011: Apache released first stable version 1.0
- 2012: Hadoop 2.0 version which contains YARN was released
- 2017: Hadoop 3.0 was released
- 2020: Hadoop 3.1.13 is the latest version of Hadoop

Hadoop Characteristics

Data is highly available and accessible despite hardware failure	High Availability
Vertical Scalability –hardware Horizontal Scalability – New nodes	Scalability
No need to purchase costly license or costly hardware	Economic
Distributed computing challenges are handled by framework	Easy to Use

Open Source	Can be redistributed Can be modified
Distributed Processing	Data is processed distributedly and independently on cluster
Fault Tolerance	Failure (hardware or tasks) of nodes are recovered automatically
Reliability	Data is reliably stored on the cluster of machines despite machine failures

Architecture Of Hadoop

hadoop MapReduce	A programming model for large scale data processing
hadoop YARN	A resource-management platform responsible for managing compute resources in clusters
hadoop HDFS	A distributed file system that provides high-throughput access to application data
Common Libraries/Utilities	Contains libraries and utilities needed by other Hadoop modules

All the modules are designed with a fundamental assumption that hardware failures are common and thus should be automatically handled in software by the framework

Hadoop Ecosystem

Ambari Provisioning, Managing and Monitoring Hadoop Clusters

- **Sqoop** Data Exchange
- **Flume** Log Collector
- **Zookeeper** Coordination
- **Oozie** Workflow
- **Pig** Scripting
- **Mahout** Machine Learning
- **R Connectors** Statistics
- **Hive** SQL Query
- **Hbase** Columnar Store
- **YARN Map Reduce v2** Distributed Processing Framework
- **HDFS** Hadoop Distributed File System

Summary and Q&A

Thank You

Q & A